

Fluxo conversacional do Cardiola

Relatório das Partes 1 e 2

Felipe Livino dos Santos | RM 563187; Daniel Veiga Rodrigues de Faria | RM 561410; Tomas Haru Sakugawa Becker | RM 564147; Daniel Tavares de Lima Freitas | RM 562625; Gabriel Konno Carrozza | RM 564468.

1 Objetivo e organização da solução

Nesta etapa do Cardiola, desenvolvemos um protótipo que recebe mensagens em linguagem natural, organiza relatos e medições fictícias e apresenta um resumo para conferência. O objetivo é demonstrar um atendimento inicial simulado com IBM Watson Assistant, backend Flask e interface HTML. A proposta não inclui diagnóstico, prescrição ou avaliação de risco. Essa delimitação orienta tanto as respostas quanto os campos que o sistema permite registrar.

O Watson conduz a conversa por intenções, entidades e nós de diálogo. O Flask faz a comunicação com a API v2, valida as mensagens e devolve ao navegador a resposta e um resumo filtrado. Escolhemos uma interface com HTML, CSS e JavaScript por ser suficiente para enviar mensagens, revisar informações e alternar entre conversa, organização de relato e monitoramento. A arquitetura acompanha os exemplos de Watson e integração web apresentados no capítulo 10 de PCV [1].

2 Como uma mensagem percorre o sistema

Ao iniciar uma conversa, o backend cria uma sessão no Watson e guarda seu identificador em SQLite. O navegador recebe um token opaco, sem carregar o relato clínico no cookie. Nas mensagens seguintes, a aplicação reutiliza a sessão remota. Assim, o assistente sabe se está aguardando uma medição, o período de um relato ou a confirmação do resumo. Perfis diferentes de navegador têm sessões separadas; abas do mesmo perfil compartilham o contexto.

A versão revisada em 7 de setembro de 2026 contém 22 intenções, 125 exemplos de treinamento, seis entidades e 69 nós. As intenções representam pedidos como registrar uma medição, corrigir um dado, pedir ajuda ou encerrar. As entidades identificam elementos úteis dentro da frase, como pressão em mmHg e frequência em bpm. Encontrar a palavra “dor”, por exemplo, não basta para afirmar que há um sintoma: o relato original é preservado, inclusive quando contém uma negação.

3 Exemplo de conversa e critérios de continuidade

Se a pessoa envia “120/80 mmHg e 72 bpm”, os dois valores são registrados na mesma interação. A resposta repete as medições e pergunta se estão corretas. Ao receber “Corrige a pressão para 125/82 mmHg”, o sistema substitui apenas a pressão, conserva a frequência e retira a confirmação anterior. A conferência precisa ser feita novamente porque o conteúdo mudou.

Também tratamos situações comuns que interrompem uma sequência rígida. “Não lembro quando começou” deixa o período sem informação; “Prefiro pular essa parte” encerra a coleta atual; um agradecimento não finaliza o atendimento. Pedidos de reformulação retomam a pergunta pendente. Essas intenções são avaliadas antes da captura de texto livre para evitar que dúvidas e recusas sejam salvas como informações de saúde.

4 Interface e tratamento de exceções

A tela mantém a conversa e o resumo próximos, permitindo comparar o que foi dito com o que ficou registrado. Os botões de sugestão acompanham a etapa: durante a coleta, oferecem reformular, pular ou cancelar; na conferência, permitem confirmar ou corrigir. A revisão visual retirou frases promocionais e elementos decorativos repetidos, simplificou a navegação e tornou o resumo semelhante a uma ficha de registro. Os avisos sobre o caráter fictício dos dados continuam visíveis.

Comandos de ajuda, correção e reinício têm prioridade sobre perguntas abertas. Uma medição sem a unidade esperada recebe orientação de formato, sem conversão por suposição.

Mensagens desconhecidas levam a um pedido de reformulação e, em caso de repetição, à apresentação de caminhos possíveis. Confirmar um resumo vazio não cria um registro. O botão Nova conversa abre outra sessão e limpa o contexto anterior; encerrar apenas conclui o fluxo, mantendo o resumo disponível.

O backend limita tamanho e tipo das requisições, verifica um token CSRF nas operações de envio e retorna erros controlados quando a sessão expira ou um serviço externo fica indisponível. As mensagens são inseridas no HTML como texto, e o resumo mostra somente campos previstos. As credenciais permanecem no arquivo local de ambiente e são injetadas nos containers. O endpoint de saúde verifica a disponibilidade HTTP, mas não comprova autenticação nas APIs.

5 Validação e resultados

Na revisão de 7 de setembro, os 27 testes automatizados do núcleo e da extração passaram. Em uma avaliação separada com o Watson real, dez cenários totalizaram 31 turnos aprovados. Foram verificados registro conjunto de medições, correção direta, recusa, reformulação, mudança de assunto, identidade do assistente e limites de atuação. A primeira rodada mostrou confusões entre confirmar e consultar o resumo e entre pedir esclarecimento e recusar uma pergunta. Os ajustes foram reavaliados após o treinamento [2].

Esses 31 turnos são uma verificação de comportamento em cenários conhecidos. Não representam compreensão irrestrita da linguagem. Uma avaliação anterior, registrada em 4 de setembro, obteve 38 classificações corretas em 42 frases, ou 90,5%, na versão antiga. Esse percentual permanece como histórico e não foi recalculado para o diálogo atual. A avaliação do código, a avaliação de intenções e a verificação da interface medem aspectos diferentes da solução.

6 Execução e conclusão

A execução completa usa “docker compose --profile automacao up --build -d”. No ambiente local revisado, a interface está na porta 5001, configurada por APP_PORT; o padrão do projeto é 5000. O protótipo atende ao fluxo de coleta, organização e conferência proposto, com limites explícitos. Ainda é necessário avaliar mais formas de expressão e reimportar o diálogo em outro ambiente antes de concluir que o comportamento se reproduz fora da configuração usada pela equipe.

[1] FIAP. Arquitetura Cognitiva dos LLMs Modernos. PCV, Fase 5, capítulo 10, p. 17 a 37 e 46 a 49. Material didático da atividade.

[2] Projeto Cardiola. docs/VALIDACAO.md; docs/evidence/conversation-evaluation.json; watson/export-provenance.json. Revisão de 7 set. 2026. Implementação: app.py, services/watson_service.py e watson/build_skill.py.